# SISTEMA DE GESTIÓN DE TAREAS
## Workable

**Documentación del Proyecto**

**Juan David Gómez Linares**
**Juan David Perez Alvarez**
**Nicolas Zequea Loboa**

**Análisis y desarrollo de software**
**SENA Bogotá, 2026**

**Control de Versiones**

| Versión | Fecha | Descripción | Autores |
|---|---|---|---|
| **1.0** | Febrero 9 de 2026 | Versión Inicial del Documento | Juan David Perez<br><br>Juan David Gomez<br><br>Nicolas Zequea |
| | | | |
| | | | |
| | | | |
| | | | |

**TABLA DE CONTENIDO**

1 OBJETIVO....4
2 ALCANCE....4
3 PÚBLICO....4
4 TÉRMINOS Y DEFINICIONES....4
5 ¿QUÉ ES EL MANUAL TÉCNICO Y DE OPERACIÓN DEL SISTEMA?....4
6 INTRODUCCIÓN....5
7 TAXONOMÍA Y CONTENIDO DEL MANUAL TÉCNICO Y DE OPERACIÓN DEL SISTEMA....6
7.1 DESCRIPCIÓN DEL SISTEMA DE INFORMACIÓN DESARROLLADO....7
7.2 DISEÑO TÉCNICO DEL SISTEMA DE INFORMACIÓN....7
6.2.1 ESQUEMA O MODELO DE REQUERIMIENTOS....8
6.2.2 SOFTWARE BASE DEL SISTEMA Y PRERREQUISITOS....8
6.2.3 COMPONENTES Y ESTÁNDARES....9
6.2.4 MODELO DE DATOS....9
6.2.5 FUNCIONALIDAD Y SERVICIOS OFRECIDOS....9
7.3 DESPLIEGUE Y CONFIGURACIÓN DE COMPONENTES QUE CONFORMAN EL SISTEMA DE INFORMACIÓN....9
6.3.1 ORGANIZACIÓN DE COMPONENTES....9
6.3.2 INSTALACIÓN....10
6.3.3 CONFIGURACIÓN....10
6.3.4 DESPLIEGUE....10
7.4 RESOLUCIÓN DE PROBLEMAS....10
6.4.1 ERRORES TÉCNICOS MÁS COMUNES Y SU SOLUCIÓN ....10

# 1. Introducción

## 1.1 Objetivo

El presente manual técnico tiene como propósito describir la arquitectura, componentes, funcionamiento y estructura interna del sistema de gestión de reclutamiento. Este documento está dirigido a desarrolladores, administradores del sistema y personal técnico encargado del mantenimiento, evolución y despliegue de la aplicación.

## 1.2 Alcance

Este manual cubre la descripción técnica completa del sistema, incluyendo:

- Arquitectura general
- Tecnologías utilizadas
- Estructura del backend y frontend
- Gestión de la base de datos
- Seguridad e implementación de autenticación
- Integración entre componentes

No incluye instrucciones de uso para usuarios finales.

## 1.3 Definición del sistema

El sistema es una plataforma web de gestión de procesos de reclutamiento que permite administrar aspirantes, vacantes y procesos de selección mediante diferentes roles: administrador, reclutador y aspirante.

## 1.4 Importancia

Este documento es fundamental para garantizar la correcta comprensión técnica del sistema, facilitando su mantenimiento, escalabilidad y futuras mejoras.

# 2. Descripción General del Sistema

## 2.1 Perspectiva del sistema

El sistema sigue una arquitectura cliente-servidor, donde el frontend consume servicios expuestos por el backend mediante una API REST.

## 2.2 Funcionalidades principales

- Gestión de usuarios y roles
- Registro y autenticación de usuarios
- Publicación y gestión de vacantes
- Postulación de aspirantes
- Seguimiento del proceso de selección

## 2.3 Usuarios del sistema

- Administrador: gestiona el sistema completo
- Reclutador: administra vacantes y procesos
- Aspirante: aplica a vacantes

## 3. Público objetivo

Este manual está dirigido a:

- Desarrolladores Full Stack
- Administradores del sistema
- Personal de soporte técnico
- Evaluadores del proyecto (SENA)

Se asume conocimiento en desarrollo web, bases de datos y arquitectura cliente-servidor.

## 4. Términos y definiciones

El presente glosario reúne los términos técnicos, acrónimos y conceptos fundamentales utilizados en el desarrollo y documentación de la plataforma Workable.

- **Navegador Web:**
  Software utilizado para acceder y visualizar la interfaz del sistema a través de Internet (ej. Chrome, Firefox, Edge).
- **Frontend:**
  Capa de presentación del sistema desarrollada en React, encargada de la interacción directa con el usuario.
- **Backend:**
  Capa de lógica de negocio desarrollada en Spring Boot (Java), responsable del procesamiento de datos, validaciones de seguridad y conexión con la base de datos.
- **API (Application Programming Interface):**
  Conjunto de endpoints que permiten la comunicación entre el Frontend y el Backend mediante peticiones HTTP y respuestas en formato JSON.
- **Endpoint:**
  Ruta específica dentro de la API donde se ejecuta una operación determinada (ej. /login, /ofertas).
- **JWT (JSON Web Token):**
  Mecanismo de autenticación basado en tokens que permite validar la identidad del usuario y mantener su sesión activa.

- **Bcrypt:**
  Algoritmo de cifrado utilizado para almacenar contraseñas de forma segura en la base de datos.

- **Base de Datos:**
  Sistema relacional (MySQL) donde se almacenan los datos persistentes de usuarios, ofertas y postulaciones.

- **XAMPP:**
  Entorno de desarrollo local que integra Apache y MySQL para pruebas del sistema.

- **phpMyAdmin:**
  Herramienta web para la administración gráfica de bases de datos MySQL, permitiendo ejecutar consultas SQL y gestionar estructuras sin usar línea de comandos.

- **Spring Boot:**
  Framework basado en Java utilizado para construir el backend del sistema, gestionar la seguridad, exponer la API REST y administrar la lógica de negocio.

- **React:**
  Librería de JavaScript utilizada para la construcción de interfaces de usuario dinámicas y responsivas.

- **Postman:**
  Herramienta para probar y validar endpoints de la API de forma independiente al frontend.

- **Control de Versiones:**
  Sistema (como Git) que permite registrar y gestionar cambios en el código fuente del proyecto.

- **CRUD:**
  Operaciones básicas sobre datos: Crear (Create), Leer (Read), Actualizar (Update) y Eliminar (Delete).

- **Rol:**
  Tipo de usuario dentro del sistema (Aspirante, Reclutador, Administrador) que determina los permisos y acciones permitidas.

- **Vacante:**
  Oferta laboral publicada por un reclutador dentro de la plataforma.

- **Postulación:**
  Acción mediante la cual un aspirante aplica a una vacante laboral.

## 5. Manual técnico y operaciones del sistema

El manual técnico y de operación de la plataforma Workable es un documento que describe de manera detallada la arquitectura, funcionamiento, instalación, configuración y mantenimiento del sistema.

Su finalidad es proporcionar una visión estructurada del sistema, permitiendo comprender su diseño, componentes y comportamiento, así como los procesos necesarios para su correcta

implementación y operación.

Incluye información sobre la arquitectura del sistema, la interacción entre sus componentes, la gestión de la base de datos, los mecanismos de seguridad y los procedimientos de despliegue y mantenimiento.

- ### 5.1 Definición

El manual técnico y de operación es un documento que describe la arquitectura, funcionamiento, instalación, configuración y mantenimiento del sistema Workable.
Su finalidad es proporcionar una visión estructurada del sistema que facilite su comprensión, implementación y gestión técnica.

- ### 5.2 Importancia

Este documento permite la transferencia de conocimiento técnico, garantizando que el sistema pueda ser operado, administrado y mantenido sin depender exclusivamente de los desarrolladores originales.

Además, respalda procesos de:

- Operación
- Administración
- Mantenimiento técnico

en cumplimiento con el Marco de Referencia de Arquitectura de MinTIC.

- ### 5.3 Orientación del contenido

El contenido del manual está estructurado para cubrir tres enfoques principales:

- **Operación:** Funcionamiento del sistema
- **Administración:** Gestión de usuarios, roles y seguridad
- **Mantenimiento:** Actualización, corrección y evolución del sistema

## 6. Taxonomía y contenido del manual técnico y de operación del sistema

Este capítulo describe la arquitectura, estructura interna y componentes técnicos que soportan el funcionamiento de la plataforma Workable, incluyendo los requerimientos, infraestructura, modelo de datos y servicios del sistema.

Tabla 1. Atributos del Manual Técnico de Workable:

| Atributo | Descripción Aplicada al Proyecto |
|---|---|
| **Prerrequisitos** | Servidores compatibles con Node.js y Java, base de datos MySQL y navegador web moderno. |
| **Frameworks y estándares** | Frontend desarrollado en React y Backend en Spring Boot, bajo arquitectura cliente-servidor. |
| **Diagrama de casos de uso** | Representa las interacciones entre los actores del sistema (Aspirantes, Reclutadores y Administradores) basadas en los requerimientos funcionales. |
| **Diagrama ER** | Modelo entidad-relación que describe la estructura de la base de datos workable_db y las relaciones entre sus entidades. |
| **Diagrama de Despliegue** | Representa la infraestructura del sistema, incluyendo frontend, backend, servidor y base de datos. |
| **Diagrama de clases** | Describe la estructura de clases del sistema, sus atributos, métodos y relaciones dentro de la lógica de negocio. |

## 6.1 DESCRIPCIÓN DEL SISTEMA DE INFORMACIÓN DESARROLLADO

Este capítulo constituye el núcleo técnico del documento, en el cual se describe la organización de los componentes del sistema Workable desde una perspectiva funcional, lógica y de despliegue.

La información se presenta de manera estructurada, iniciando desde los requerimientos funcionales del sistema hasta los elementos técnicos que soportan su implementación,

incluyendo la arquitectura, modelos de datos y diagramas de diseño.

## 6.2 DISEÑO TÉCNICO DEL SISTEMA DE INFORMACIÓN

Esta sección detalla los elementos técnicos que permiten la implementación del sistema, incluyendo requerimientos, infraestructura, estándares, modelo de datos y servicios backend.

### 6.2.1 ESQUEMA O MODELO DE REQUERIMIENTOS

Los requerimientos funcionales de la plataforma Workable (RF01–RF09) definen el comportamiento del sistema y estructuran su arquitectura, módulos y servicios backend.

Estos se agrupan en los siguientes bloques funcionales:

**Matriz de Trazabilidad: Épicas y Requerimientos**

| Épica | RF | Roles | Descripción |
|---|---|---|---|
| **Autenticación y Seguridad** | **RF01, RF02** | **Aspirante, Reclutador, Admin** | **Registro de usuarios y autenticación mediante JWT** |
| **Gestión de Perfiles** | **RF03, RF04** | **Aspirante, Reclutador, Admin** | **Administración y actualización de datos de usuario** |
| **Gestión de Reclutadores** | **RF05** | **Admin** | **Creación y gestión de cuentas de reclutador** |
| **Gestión de Ofertas Laborales** | **RF06, RF07** | **Reclutador, Admin** | **Creación, edición y eliminación de vacantes** |

| | | | |
|---|---|---|---|
| **Búsqueda y Postulación** | **RF08, RF09** | **Aspirante** | **Consulta de ofertas y aplicación a vacantes** |

### 6.2.2 Software base del sistema y prerrequisitos

Esta sección describe el software base y los requerimientos de infraestructura necesarios para la implementación y ejecución de la plataforma Workable.

#### 6.2.2.1 Servidores de Aplicaciones (Frontend)

Esta capa gestiona la interacción con el usuario final.

**Software base:**

- Entorno de ejecución: Node.js (v22.14.0)
- Framework: React
- Servidor web: Apache HTTP Server 2.4 (XAMPP)

**Protocolos y puertos:**
- Desarrollo: 3000 / 5173
- Producción: 80 / 443 (HTTP/HTTPS)

**Requerimientos de hardware:**
- CPU: mínimo 1 core (2.0 GHz)
- RAM: 2 GB

#### 6.2.2.2 SERVIDORES DE NEGOCIO (BACKEND)

Esta capa contiene la lógica de negocio, seguridad y procesamiento del sistema.
**Software base:**
- Lenguaje: Java 21 (LTS)
- Framework: Spring Boot 3.x
- Seguridad: Spring Security + JWT + BCrypt
- Gestión del proyecto: Apache Maven

**Arquitectura interna:**
- Controllers
- Services
- Repositories
- Entities

**Comunicación:**
- API REST sobre HTTP/1.1
- Puerto: 8080
- CORS habilitado (frontend autorizado)

**Requerimientos de hardware:**

- RAM: 4 GB

---

#### 6.2.2.3 Servidores de Base de Datos

Encargados del almacenamiento persistente del sistema.

**Tecnología:**
- Motor: MySQL / MariaDB
- ORM: JPA + Hibernate

**Configuración:**
- Estrategia: update/create
- Inicialización: carga de datos maestros (municipios)

**Administración:**
- Herramienta: phpMyAdmin

**Requerimientos:**
- RAM: 1 GB
- Disco: 1 GB inicial

### 6.2.3 Componentes y estándares

Esta sección define los estándares técnicos y de desarrollo aplicados en la construcción del sistema Workable.

#### 6.2.3.1 Estandares de arquitectura y comunicacion

El sistema implementa una arquitectura basada en separación de responsabilidades (SoC), dividida en frontend y backend.

- Estilo arquitectónico: REST
- Formato de datos: JSON
- Protocolo: HTTP/1.1

#### 6.2.3.2 Estandares de codificacion y desarrollo

**Frontend:**

- Uso de React con Hooks
- Componentes reutilizables
- Convenciones: PascalCase / camelCase

**Backend:**

- Arquitectura por capas (Controller, Service, Repository)
- Uso de Lombok
- Validaciones con Jakarta Validation

#### 6.2.3.3 Seguridad y control de acceso

El sistema implementa seguridad basada en roles alineada a los RF:

- Autenticación mediante JWT
- Encriptación de contraseñas con BCrypt
- Control de acceso por roles (Aspirante, Reclutador, Admin)
- Configuración CORS restringida al frontend

### 6.2.4 Modelo de datos

Describe la estructura de almacenamiento del sistema y su relación con los requerimientos funcionales.

#### 6.2.4.1 MODELO ENTIDAD-RELACION (LOGICO Y FISICO

El sistema utiliza un modelo relacional implementado en MySQL, diseñado para soportar los requerimientos funcionales (RF01–RF09).

A nivel lógico, los datos se organizan en:

- Usuarios (autenticación y perfiles)
- Empresas/Reclutadores
- Ofertas laborales
- Postulaciones

A nivel físico, se implementa mediante tablas relacionadas con integridad referencial, utilizando claves primarias y foráneas para garantizar consistencia de datos.

#### 6.2.4.2 DICCIONARIO DE DATOS

A continuación, se detallan las tablas principales que conforman la base de datos **workable_db**, ajustadas a los requerimientos funcionales actuales (RF01–RF09) y a la gestión de roles incluyendo **Administrador**:

| Tabla | Campo | Tipo de Dato | Restricción | Descripción |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| **USUARIOS** | id | BIGINT | PK, AI | Identificador único del usuario. |
| | email | VARCHAR(255) | Unique, NN | Correo electrónico utilizado para autenticación |
| | password | VARCHAR(255) | NN | Contraseña encriptada con BCrypt |
| | rol | ENUM | NN | Valores: ASPIRANTE, RECLUTADOR, ADMIN. |
| **EMPRESA** | id | BIGINT | PK, AI | Identificador único de la empresa. |
| | nit | VARCHAR(20) | Unique, NN | Número de identificación tributaria. |
| | nombre | VARCHAR(100) | NN | Nombre comercial de la empresa. |
| | usuario_id | BIGINT | FK | Relación con usuario tipo RECLUTADOR |

| | | | | |
|---|---|---|---|---|
| **OFERTA** | id | BIGINT | PK, AI | Identificador de la vacante. |
| | empresa_id | BIGINT | FK | Empresa que publica la oferta |
| | titulo | VARCHAR(150) | NN | Nombre del cargo |
| | estado | VARCHAR(20) | NN | Estado: ABIERTA, CERRADA. |
| | descripción | TEXT | NN | Descripción de la vacante |
| | creado_por | BIGINT | FK | Usuario (RECLUTADOR o ADMIN) que crea la oferta |
| **POSTULACIÓN** | id | BIGINT | PK, AI | Identificador de la postulación |
| | aspirante_id | BIGINT | FK | Usuario que aplica |
| | oferta_id | BIGINT | FK | Vacante aplicada |

| | | | | |
|---|---|---|---|---|
| | estado_id | VARCHAR(20) | NN | PENDIENTE, ACEPTADO, RECHAZADO |

#### 6.2.4.3 OBJETOS EMBEBIDOS Y PERSISTENCIA ESPECIAL

El sistema optimiza la gestión de la información curricular mediante el uso de **objetos embebidos**, reduciendo la complejidad de consultas y mejorando el rendimiento.

- **EstudioData:**
  Contiene nivel educativo, institución y fechas de formación.
- **ExperienciaData:**
  Almacena historial laboral del aspirante, incluyendo empresa, cargo y duración.
- **CitaciónData:**
  Gestiona la programación de entrevistas, incluyendo fecha, estado y observaciones del proceso.

#### 6.2.4.4 ESPECIFICACIONES TECNICAS DEL ALMACENAMIENTO

- **Motor de almacenamiento:** InnoDB
  (Soporte de transacciones y claves foráneas)

- **Cotejamiento:** utf8mb4_unicode_ci
  (Compatibilidad completa con caracteres especiales)

- **Estrategia de generación de IDs:**
  Implementada mediante Hibernate con:

  @GeneratedValue(strategy = GenerationType.IDENTITY)

### 6.2.5 FUNCIONALIDAD Y SERVICIOS OFRECIDOS

La plataforma Workable expone un conjunto de servicios backend estructurados bajo arquitectura REST, alineados con los requerimientos funcionales (RF01–RF09) y los roles del sistema.

## Controladores principales:

- **AuthController**

  - Registro de usuarios (Aspirante, Reclutador)
  - Autenticación con JWT
    (RF01, RF02)

- **UsuarioController**

  - Gestión de perfiles
  - Actualización de información personal
    (RF03, RF04)

- **AdminController**
    - Creación y gestión de reclutadores
    - Supervisión del sistema (RF05)

- **OfertaController**
    - Crear, editar y eliminar ofertas laborales
    - Gestión de estado de vacantes (RF06, RF07)
- **PostulacionController**
    - Aplicación a ofertas
    - Consulta de postulaciones (RF08, RF09)

#### 6.2.5.1 REGLAS DE NEGOCIO Y LOGICA DE SERVICIOS

Los servicios expuestos por la plataforma Workable no se limitan a operaciones básicas tipo CRUD, sino que incorporan reglas de negocio, validaciones de seguridad y control de acceso según el rol del usuario.

Las principales reglas implementadas son:

**Validación de Postulación Única:**
El sistema garantiza que un aspirante no pueda postularse más de una vez a la misma vacante. Adicionalmente, se valida que la oferta se encuentre en estado **"ABIERTA"** antes de permitir la postulación.

**Control de Acceso por Roles:**
El acceso a los endpoints está restringido mediante anotaciones de seguridad (por ejemplo, `@PreAuthorize`), asegurando que cada funcionalidad sea ejecutada únicamente por el rol correspondiente:

- **Aspirante:** consulta de ofertas y postulaciones.
- **Reclutador:** creación y gestión de vacantes.
- **Administrador:** gestión de reclutadores, control de ofertas y supervisión del sistema.

**Gestión Administrativa del Sistema:**
El rol administrador tiene la capacidad de crear cuentas de reclutadores, eliminar ofertas que incumplan las políticas y supervisar la información del sistema, garantizando el control y la integridad de la plataforma.

**Validaciones de Datos:**
Se aplican validaciones a nivel de backend para asegurar la integridad de la información (formatos de

correo, campos obligatorios, rangos de datos, entre otros), evitando el ingreso de datos inconsistentes.

**Manejo Centralizado de Excepciones:**
Se implementa un manejador global de errores (*Global Exception Handler*) que captura excepciones del sistema (como credenciales inválidas, recursos no encontrados o token expirado) y retorna respuestas estructuradas en formato JSON al frontend.

#### 6.2.5.2 SERVICIOS DE CONFIGURACION Y CORS

El sistema cuenta con una configuración de interoperabilidad que permite la comunicación segura entre el frontend y el backend.

**Configuración CORS (Cross-Origin Resource Sharing):**
Se definen políticas de acceso que permiten solicitudes únicamente desde el origen autorizado del frontend (por ejemplo, `http://localhost:5173`), evitando accesos no permitidos desde otros dominios.

**Gestión de Autenticación con JWT:**
Las peticiones a los servicios protegidos requieren el envío de un token JWT en la cabecera `Authorization`. Este token es validado en cada solicitud para garantizar la autenticidad del usuario y su rol dentro del sistema.

**Configuración de Seguridad:**
Se establecen filtros de seguridad mediante Spring Security que interceptan las solicitudes HTTP, validan credenciales y controlan el acceso a los recursos protegidos.

## 6.3 DESPLIEGUE Y CONFIGURACIÓN DE COMPONENTES DEL SISTEMA

---

### 6.3.1 ORGANIZACIÓN DE COMPONENTES

El sistema Workable está estructurado bajo una arquitectura cliente-servidor, donde el frontend interactúa con el backend mediante servicios REST, y este último gestiona la lógica de negocio y la persistencia de datos.

La organización del sistema se compone de los siguientes elementos:

**Frontend Web (React):**
Aplicación cliente que se ejecuta en el navegador. Permite la interacción con los usuarios del sistema (registro, inicio de sesión, consulta de ofertas y postulación).**Aplicación Móvil (TypeScript):**

Cliente móvil que consume los mismos servicios del backend, permitiendo el acceso a funcionalidades principales desde dispositivos móviles.

**Backend (Spring Boot):**
Servidor encargado de procesar las solicitudes, aplicar las reglas de negocio, gestionar la seguridad

(JWT y roles) y exponer los servicios REST.

**Base de Datos (MySQL):**
Sistema de almacenamiento que gestiona la persistencia de la información, incluyendo usuarios, empresas, ofertas y postulaciones.

A nivel estructural, el proyecto se organiza en módulos independientes (frontend y backend), facilitando su mantenimiento, escalabilidad y despliegue.

---

### 6.3.2 INSTALACIÓN

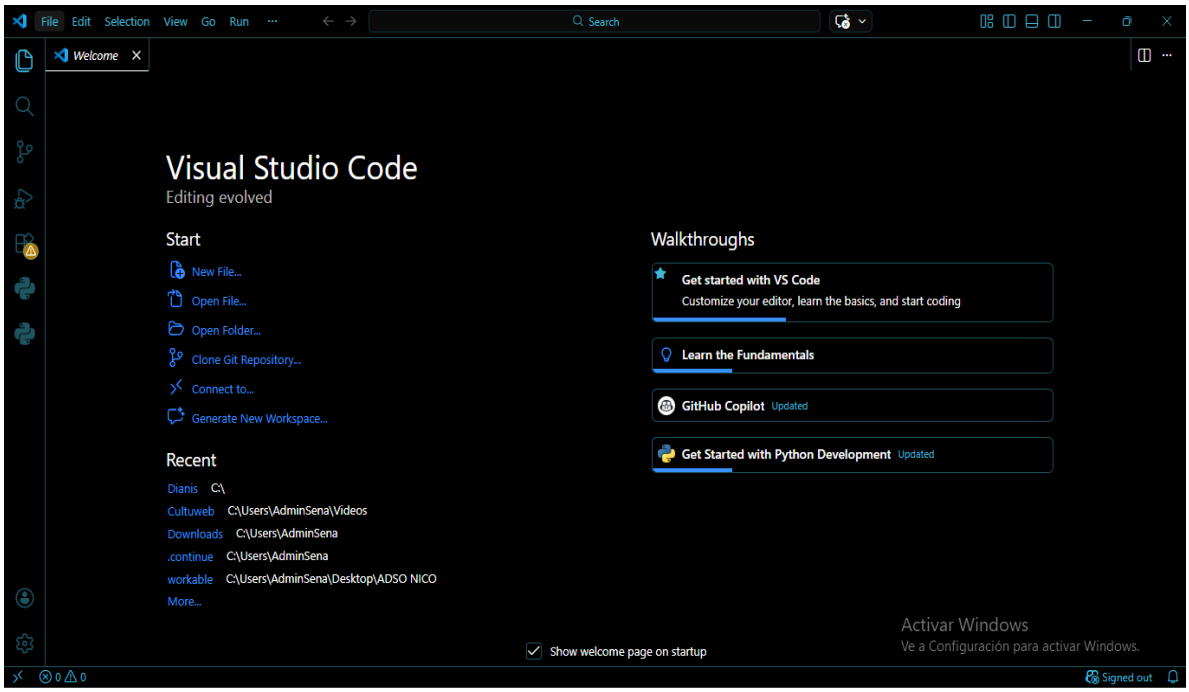
Para la ejecución del sistema Workable, es necesario contar con un entorno de desarrollo correctamente configurado. A continuación, se describen los pasos detallados para la instalación de cada herramienta requerida.

#### Instalación de Visual Studio Code

Se utilizará Visual Studio Code como editor principal de desarrollo.

**Pasos:**

1. Ingresar al sitio oficial: https://code.visualstudio.com
2. Descargar la versión correspondiente al sistema operativo.
3. Ejecutar el instalador descargado.
4. Durante la instalación se recomienda marcar las siguientes opciones:
   - Add to PATH
   - Crear acceso directo en el escritorio
5. Abrir con Code
6. Finalizar la instalación.

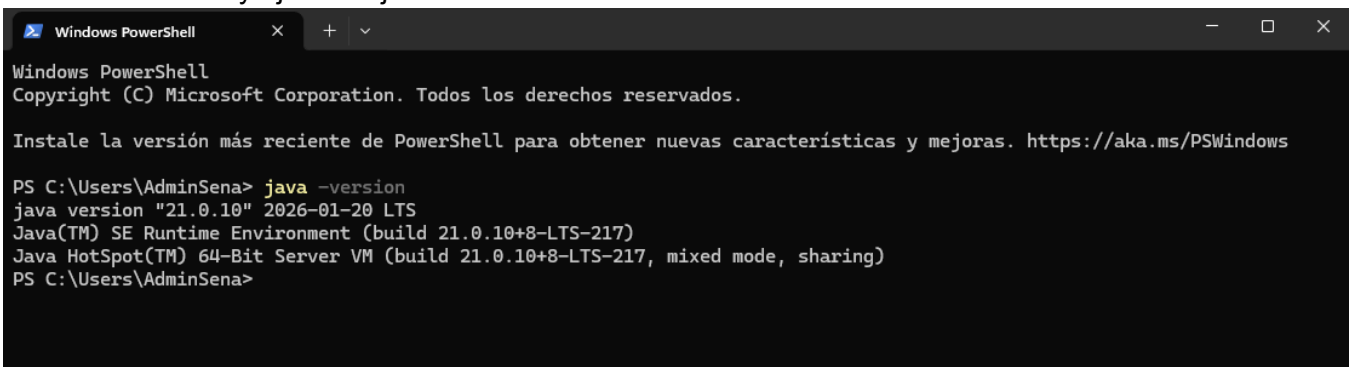


**Instalar el entorno de Java**

1. Ingresar al sitio oficial de Java: https://www.oracle.com/java/technologies/downloads/
2. Descargar la versión **JDK 21 para Windows x64**.
3. Ejecutar el instalador descargado.
4. Seguir los pasos del asistente de instalación.
5. Finalizar la instalación.

| | | |
|---|---|---|
| Windows x64 Compressed Archive | 186.13 MB | https://download.oracle.com/java/21/archive/jdk-21.0.9_windows-x64_bin.zip (sha256) |
| Windows x64 Installer | 164.45 MB | https://download.oracle.com/java/21/archive/jdk-21.0.9_windows-x64_bin.exe (sha256) |
| Windows x64 msi Installer | 163.20 MB | https://download.oracle.com/java/21/archive/jdk-21.0.9_windows-x64_bin.msi (sha256) |

Si la instalación fue correcta, se mostrará la versión de Java instalada.
Instalar el Windows x64 Installer de 164.45MB
Abrir una terminal y ejecutar: java -version




### 6.3.3 CONFIGURACIÓN

Una vez instalado el entorno, se debe configurar la comunicación entre los componentes del sistema.

**Configuración del Backend:**
En el archivo application.properties o application.yml se deben definir:

- URL de la base de datos
- Usuario y contraseña
- Puerto del servidor (por defecto 8080)

**Configuración del Frontend:**
Se debe establecer correctamente la URL del backend, por ejemplo:
http://localhost:8080

**Configuración de Seguridad:**
Se deben verificar las configuraciones de CORS y JWT para permitir la comunicación segura entre frontend y backend.

---

### 6.3.4 DESPLIEGUE

El despliegue consiste en la ejecución coordinada de todos los componentes del sistema.

**Pasos para el despliegue en entorno local:**

1. Iniciar el servicio de MySQL
2. Ejecutar el backend (Spring Boot)
3. Ejecutar el frontend con:
   npm run dev
4. Acceder al sistema desde el navegador
5. Verificar funcionalidades como registro, autenticación y gestión de ofertas

Una vez todos los servicios están activos, el sistema se encuentra operativo en entorno local.

---

## 6.4 RESOLUCIÓN DE PROBLEMAS

---

### 6.4.1 ERRORES TÉCNICOS MÁS COMUNES Y SU SOLUCIÓN

**1. Error de conexión a la base de datos**

- *Causa:* Configuración incorrecta en el backend
- *Solución:* Verificar credenciales, URL y nombre de la base de datos

**2. El backend no inicia**

- *Causa:* Puerto ocupado o error en dependencias
- *Solución:* Cambiar el puerto o revisar el archivo pom.xml

**3. El frontend no se conecta al backend**

- *Causa:* URL mal configurada

- *Solución:* Verificar la dirección del backend en el frontend

**4. Error al ejecutar npm install**

- *Causa:* Problemas de red o incompatibilidad de versiones
- *Solución:* Verificar conexión a internet y versión de Node.js

**5. Problemas de autenticación (JWT)**

- *Causa:* Token inválido o expirado
- *Solución:* Iniciar sesión nuevamente y validar configuración de seguridad